JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Annotation Interface SuppressWarnings

```
@Retention(SOURCE)
public @interface SuppressWarnings
```

Indicates the warnings to be suppressed at compile time in the annotated element, and in all elements contained in the annotated element.

The `SuppressWarnings` annotation interface is applicable in all declaration contexts, so an `@SuppressWarnings` annotation can be used on any element. As a matter of style, programmers should always use this annotation on the most deeply nested element where it is effective. For example, if you want to suppress a warning in a particular method, you should annotate that method rather than its class.

The set of warnings suppressed in a given element is a union of the warnings suppressed in all containing elements. For example, if you annotate a class to suppress one warning and annotate a method in the class to suppress another, both warnings will be suppressed in the method. However, note that if a warning is suppressed in a `module-info` file, the suppression applies to elements within the file and *not* to types contained within the module. Likewise, if a warning is suppressed in a `package-info` file, the suppression applies to elements within the file and *not* to types contained within the package.

Java compilers must recognize all the kinds of warnings defined in the *Java Language Specification* (JLS section §9.6.4.5) which include:

- Unchecked warnings, specified by the string `"unchecked"`.
- Deprecation warnings, specified by the string `"deprecation"`.
- Removal warnings, specified by the string `"removal"`.
- Preview warnings, specified by the string `"preview"`.

Whether or not a Java compiler recognizes other strings is a quality of implementation concern. Compiler vendors should document the additional warning names they support. Vendors are encouraged to cooperate to ensure that the same names work across multiple compilers.

Implementation Note:

In addition to the mandated suppression strings, the `javac` reference implementation recognizes compilation-related warning names documented in its `--help-lint` output.

See *Java Language Specification*:

4.8 Raw Types
4.12.2 Variables of Reference Type
5.1.9 Unchecked Conversion

5.5 Casting Contexts
9.6.4.5 @SuppressWarnings

Since:

1.5

Required Element Summary

Required Elements

Modifier and Type	Required Element	Description
`String[]`	**`value`**	The set of warnings that are to be suppressed by the compiler in the annotated element.

Element Details

value

`String[] value`

The set of warnings that are to be suppressed by the compiler in the annotated element. Duplicate names are permitted. The second and successive occurrences of a name are ignored. The presence of unrecognized warning names is *not* an error: Compilers must ignore any warning names they do not recognize. They are, however, free to emit a warning if an annotation contains an unrecognized warning name.

Returns:

the set of warnings to be suppressed

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.